

ACID SYNTH ESP32

Sintetizador estilo TB-303 con ESP32

Documentación técnica completa · Hardware + Firmware + Síntesis

Versión 1.0 · 2025

Oscilador VCO Sawtooth / Square · Filtro paso-bajo 24dB/oct (Moog-like) · Envolvente VCA+VCF ·
Secuenciador 16 pasos

Accent · Slide / Portamento · LFO · Salida I2S 16-bit · Control MIDI USB

1. Descripción del Proyecto

El ACID SYNTH ESP32 es una réplica funcional del sonido del Roland TB-303 Bass Line, construida sobre un microcontrolador ESP32. El TB-303 original (1981) es la base del sonido acid techno gracias a su filtro de escalera analógico con resonancia y su secuenciador de patrones característico. Este proyecto implementa todas esas características en forma digital, usando síntesis por tabla de ondas y modelado de filtro digital (IIR ladder filter).

1.1 Características principales

- Oscilador principal (VCO): sawtooth, square, triangle — seleccionable
- Sub-oscilador: 1 octava abajo, square wave
- Filtro Ladder 24dB/oct con resonancia (Q) hasta auto-oscilación
- Envolvente dedicada para VCF (cutoff) y VCA (amplitud)
- Accent: sube volumen + cutoff en notas marcadas
- Slide / Portamento: deslizamiento de pitch entre notas
- LFO: modulación de cutoff, pitch o amplitud
- Secuenciador de 16 pasos con tempo BPM ajustable
- Salida de audio I2S (DAC externo PCM5102A)
- Control MIDI vía USB Serial o jack TRS
- 8 potenciómetros analógicos para parámetros en tiempo real
- Display OLED 128×64 para visualización de parámetros

1.2 El sonido ACID — cómo funciona

El secreto del sonido acid está en la interacción de tres elementos:

⚡ **Filtro ladder con resonancia alta (70-100%) + envolvente corta de VCF + Accent en notas seleccionadas = el 'squelch' característico**

1. Sawtooth wave: rica en armónicos, es la base del timbre.
2. Filtro paso-bajo con cutoff bajo (~200-800 Hz) y resonancia alta: recorta agresivamente los armónicos y crea un pico en la frecuencia de corte.
3. Envolvente VCF con DECAY corto (50-200ms) + SUSTAIN bajo: el filtro se abre y cierra rápidamente en cada nota.
4. Accent: notas con accent tienen +12dB de volumen y el filtro se abre más, generando el 'squeak' acid.
5. Slide: las notas con slide se deslizan en pitch usando portamento, creando el efecto glide del 303.

2. Lista de Materiales (BOM)

2.1 Componentes principales

Componente	Valor / Modelo	Cantidad	Precio Aprox.	Notas
ESP32 DevKit v1	38 pines	1	\$4–8 USD	NodeMCU-32S o similar
DAC I2S	PCM5102A breakout	1	\$3–6 USD	Salida estéreo 32-bit
Display OLED	SSD1306 128×64 I2C	1	\$2–4 USD	
Potenciómetro 10kΩ	Lin B10K	8	\$0.30 c/u	Para parámetros
Potenciómetro 10kΩ	Log A10K	2	\$0.30 c/u	Para volumen/resonancia
Encoder rotativo	KY-040	1	\$1–2 USD	Para BPM/menú
Botón pulsador	12mm momentáneo	8	\$0.10 c/u	Para notas del secuenciador
Botón pulsador	12mm momentáneo	4	\$0.10 c/u	Play/Stop/Record/Shift
LED 3mm	Verde / Rojo / Amarillo	18	\$0.05 c/u	Step indicators
Conector TRS 3.5mm	Hembra PCB	2	\$0.50 c/u	Salida audio + MIDI
Conector MIDI DIN5	Hembra PCB	1	\$0.80 USD	MIDI IN (opcional)
Jack DC Barrel	5.5/2.1mm hembra	1	\$0.30 USD	Alimentación

2.2 Componentes pasivos

Componente	Valor	Cantidad	Uso
Resistencia	10 kΩ 1/4W	20	Pull-ups, divisores
Resistencia	220 Ω 1/4W	20	LEDs
Resistencia	1 kΩ 1/4W	10	General
Resistencia	33 Ω 1/4W	2	Filtro de salida audio
Condensador cerámico	100 nF (104)	10	Bypass
Condensador electrolítico	10 μF 16V	4	Bypass / acoplamiento
Condensador electrolítico	100 μF 16V	2	Decoupling alimentación
Optoacoplador	6N138 o PC817	1	MIDI IN

Componente	Valor	Cantidad	Uso
Diodo	1N4148	2	Protección MIDI

2.3 Herramientas y materiales de construcción

- PCB perforada universal 10×15 cm (o diseño PCB personalizado)
- Cautín de punta fina + estaño sin plomo 0.6mm
- Cable flexible 22 AWG (varios colores)
- Carcasa / enclosure: caja de aluminio o impresión 3D (~200×120×50mm)
- Regulador de voltaje 3.3V (AMS1117-3.3) si usas fuente 5V/12V
- Conector USB micro/tipo-C para programación
- Cable USB para programación

⚡ **Costo total estimado del proyecto: \$25–45 USD con PCB perforada. Con PCB fabricada profesionalmente: \$45–70 USD.**

3. Esquemático y Conexiones

3.1 Pinout del ESP32 (mapa completo)

Pin ESP32	GPIO	Función	Conectado a
GPIO 25	25	I2S – LRCK (Word Select)	PCM5102A – LCK
GPIO 26	26	I2S – BCLK (Bit Clock)	PCM5102A – BCK
GPIO 22	22	I2S – DOUT (Data)	PCM5102A – DIN
GPIO 21	21	I2C – SDA	OLED SDA
GPIO 22 (alt)	22	I2C – SCL	OLED SCL (si no usa I2S)
GPIO 19	19	I2C – SCL (usar este)	OLED SCL
GPIO 34	34	ADC – Potenciómetro 1	POT Cutoff Frequency
GPIO 35	35	ADC – Potenciómetro 2	POT Resonance
GPIO 32	32	ADC – Potenciómetro 3	POT Envelope Decay
GPIO 33	33	ADC – Potenciómetro 4	POT Accent Level
GPIO 36	36	ADC – Potenciómetro 5	POT Volume
GPIO 39	39	ADC – Potenciómetro 6	POT Waveform blend
GPIO 4	4	ADC – Potenciómetro 7	POT LFO Rate
GPIO 2	2	ADC – Potenciómetro 8	POT Slide Time
GPIO 12–19	12-19	Botones steps 1-8	Pulsadores step (con pull-up)
GPIO 5	5	Botón PLAY	Pulsador Play/Stop
GPIO 18	18	Encoder CLK	KY-040 CLK
GPIO 17	17	Encoder DT	KY-040 DT
GPIO 16	16	Encoder SW	KY-040 SW (click)
GPIO 1 (TX0)	1	UART TX	MIDI OUT (via 220Ω)
GPIO 3 (RX0)	3	UART RX	MIDI IN (via optoacoplador)
GPIO 13	13	LED status	LED indicador beat
3V3	—	VCC 3.3V	VCC todos los módulos
GND	—	Tierra	GND común

⚡ El ESP32 tiene dos núcleos. El núcleo 0 se dedicará a la generación de audio (I2S DMA). El núcleo 1 al secuenciador, lectura de pots y UI. Esto evita glitches de audio.

3.2 Conexión del DAC PCM5102A

El PCM5102A es un DAC I2S estéreo de alta calidad (112dB SNR). Conexiones:

Pin PCM5102A	Conectar a	Notas
VCC	3.3V ESP32	
GND	GND	
LCK (LRCK)	GPIO 25	Word Select / Frame Clock
BCK (BCLK)	GPIO 26	Bit Clock
DIN	GPIO 22	Serial Data
SCK	GND	Sistema clock — poner a GND en modo esclavo
FMT	GND	Formato I2S estándar
XSMT	3.3V	Deshabilitar mute (activo bajo)
LOUT	Jack L (via 33Ω)	Salida canal izquierdo
ROUT	Jack R (via 33Ω)	Salida canal derecho
VOUT	Capacitor 10μF a GND	Voltaje de referencia — no conectar a carga

3.3 Circuito MIDI IN

```
MIDI IN (DIN5 pin 4) — 220Ω — Ánodo Optoacoplador (6N138)
MIDI IN (DIN5 pin 5) — Cátodo Optoacoplador
                      + Diodo 1N4148 en paralelo (inversamente)
Optoacoplador VCC (pin 8) — 3.3V
Optoacoplador GND (pin 5) — GND
Optoacoplador OUT (pin 6) — 4.7kΩ pull-up a 3.3V — GPIO 3 (RX)
```

3.4 Divisor resistivo para potenciómetros

Los ADC del ESP32 operan a 3.3V máximo. Los potenciómetros se conectan directamente:

```
3.3V — Pin 1 del Pot —
      Pin 2 (wiper) — GPIO 34/35/32/33...
      Pin 3 — GND

Valor: 10kΩ lineal para todos los parámetros de síntesis
Nota: El ADC del ESP32 no es perfectamente lineal. Usar tabla de
      corrección o filtro de media móvil (ver código).
```

4. Teoría de Síntesis — Cómo suena el TB-303

4.1 El oscilador (VCO)

El VCO del 303 original usa un oscilador sawtooth (diente de sierra). La onda sawtooth contiene todos los armónicos (fundamental + 2^a + 3^a + 4^a ...) con amplitud decreciente ($1/n$), lo que la hace ideal para síntesis substractiva.

En nuestro ESP32, generamos la onda con tabla de ondas (wavetable) de 2048 muestras a 44100 Hz. La frecuencia se controla con un acumulador de fase de 32 bits:

```
phase_increment = (frecuencia * WAVETABLE_SIZE) / SAMPLE_RATE
// Para A4 (440 Hz): increment = (440 * 2048) / 44100 ≈ 20.42
phase_accumulator += phase_increment // actualizar por muestra
sample = wavetable[phase_accumulator >> 21] // leer tabla
```

4.2 El filtro Ladder (Moog-like)

Este es el corazón del sonido acid. El filtro de escalera de Moog es un filtro paso-bajo de 4 polos (24 dB/octava) con realimentación de resonancia. El modelo digital más común es el filtro de Huovilainen (2004) o el de Stilson/Smith.

Implementamos el modelo Huovilainen 0-delay feedback ladder:

```
// Parámetros:
// cutoff: 0.0 - 1.0 (normalizado a frecuencia de corte)
// resonance: 0.0 - 1.0 (0 = sin resonancia, >0.8 = auto-oscilación)

float fc = cutoff * 0.5f; // Nyquist normalizado
float f = fc * 1.16f;
float k = resonance * 4.0f; // feedback gain
float p = f * (1.8f - 0.8f * f);
float q = k * (1.0f + 0.5f * p * (1.0f - p));

// Bucle de 4 etapas (por muestra):
input -= q * stage[3]; // feedback
input = tanh(input); // saturación suave
stage[0] = input * p + delay[0] * (1.0f - p);
stage[1] = stage[0] * p + delay[1] * (1.0f - p);
stage[2] = stage[1] * p + delay[2] * (1.0f - p);
stage[3] = stage[2] * p + delay[3] * (1.0f - p);
output = stage[3];
```

⚡ La función `tanh()` es crítica — simula la saturación no-lineal de los transistores del 303 original. Sin ella el sonido es más 'digital' y menos orgánico.

4.3 Envolvente ADSR para el filtro (VCF Env)

El 303 tiene una envolvente simplificada: solo DECAY y SUSTAIN (attack fijo, release = decay). Para el sonido acid, se usa:

- Attack: 1–5 ms (muy rápido)
- Decay: 50–300 ms (el parámetro más importante del acid)
- Sustain: 0–30% (bajo, deja caer el filtro)
- Release: igual que Decay

La envolvente controla el CUTOFF del filtro: $\text{cutoff_actual} = \text{cutoff_base} + \text{envolvente} * \text{env_depth}$

4.4 Accent y Slide

ACCENT: Cuando un step tiene accent activado, se aplican dos cambios: +12 dB de amplitud (VCA) y el cutoff del filtro sube adicional ~2 octavas. Esto crea el 'squeak' característico del 303.

SLIDE: Cuando un step tiene slide activo, el pitch NO salta instantáneamente al siguiente step — se desliza usando portamento. La velocidad del slide es configurable (típicamente 30–120 ms).

```
// Portamento (slide) — interpolación exponencial:
if (slide_active) {
    current_freq += (target_freq - current_freq) * slide_rate;
    // slide_rate ≈ 0.05 - 0.20 (más alto = más rápido)
} else {
    current_freq = target_freq; // salto instantáneo
}
```


5. Firmware — Código Completo ESP32

El proyecto usa Arduino framework sobre ESP32. Instala las siguientes librerías desde el Library Manager de Arduino IDE:

- ESP32 Arduino Core v2.x (Espressif)
- Adafruit SSD1306 + Adafruit GFX Library
- No se requieren librerías externas para I2S (está en el core ESP32)

5.1 Estructura de archivos del proyecto

```
acid_synth_esp32/  
├── acid_synth_esp32.ino    ← Archivo principal (setup/loop)  
├── synth.h                ← Declaraciones del sintetizador  
├── synth.cpp              ← Motor de síntesis (VCO + VCF + VCA)  
├── sequencer.h            ← Declaraciones del secuenciador  
├── sequencer.cpp          ← Lógica del secuenciador de 16 pasos  
├── midi.h                 ← Manejo MIDI  
├── midi.cpp               ← Parser MIDI + handlers  
├── ui.h                   ← UI y display  
└── ui.cpp                 ← OLED + potenciómetros + botones
```

5.2 Archivo principal: acid_synth_esp32.ino

```
#include "Arduino.h"  
#include "driver/i2s.h"  
#include "synth.h"  
#include "sequencer.h"  
#include "midi.h"  
#include "ui.h"  
  
// — Configuración I2S —  
#define I2S_NUM          I2S_NUM_0  
#define I2S_BCK_PIN      26  
#define I2S_WS_PIN       25  
#define I2S_DATA_PIN     22  
#define SAMPLE_RATE      44100  
#define BUFFER_SIZE      256    // muestras por buffer DMA  
  
static int16_t audioBuffer[BUFFER_SIZE * 2]; // stereo  
Synth synth;  
Sequencer seq;  
MidiHandler midi;  
UI ui;  
  
// — Tarea de audio (Núcleo 0) —  
void audioTask(void* param) {  
    size_t bytesWritten;  
    while(true) {  
        synth.process(audioBuffer, BUFFER_SIZE);  
        i2s_write(I2S_NUM, audioBuffer, BUFFER_SIZE * 4, &bytesWritten, portMAX_DELAY);  
    }  
}  
  
void setup() {  
    Serial.begin(31250); // MIDI baud rate  
    synth.init();  
    seq.init(&synth);
```

```

midi.init(&synth, &seq);
ui.init(&synth, &seq);

// Configurar I2S
i2s_config_t i2s_config = {
    .mode = (i2s_mode_t)(I2S_MODE_MASTER | I2S_MODE_TX),
    .sample_rate = SAMPLE_RATE,
    .bits_per_sample = I2S_BITS_PER_SAMPLE_16BIT,
    .channel_format = I2S_CHANNEL_FMT_RIGHT_LEFT,
    .communication_format = I2S_COMM_FORMAT_STAND_I2S,
    .intr_alloc_flags = ESP_INTR_FLAG_LEVEL1,
    .dma_buf_count = 8,
    .dma_buf_len = BUFFER_SIZE,
    .use_apll = true, // Mejor jitter con APLL
    .tx_desc_auto_clear = true
};
i2s_pin_config_t pin_config = {
    .bck_io_num = I2S_BCK_PIN,
    .ws_io_num = I2S_WS_PIN,
    .data_out_num = I2S_DATA_PIN,
    .data_in_num = I2S_PIN_NO_CHANGE
};
i2s_driver_install(I2S_NUM, &i2s_config, 0, NULL);
i2s_set_pin(I2S_NUM, &pin_config);

// Lanzar tarea de audio en Núcleo 0, prioridad máxima
xTaskCreatePinnedToCore(audioTask, 'Audio', 8192, NULL, 24, NULL, 0);
}

void loop() {
    // Núcleo 1: secuenciador + UI + MIDI
    seq.update();
    midi.update();
    ui.update();
    vTaskDelay(1); // yield al OS
}

```

5.3 synth.h — Declaraciones del motor de síntesis

```

#pragma once
#include <stdint.h>
#include <math.h>

#define WAVETABLE_SIZE 2048
#define SAMPLE_RATE 44100
#define TWO_PI 6.28318530f

enum WaveShape { WAVE_SAW, WAVE_SQUARE, WAVE_TRI };

struct SynthParams {
    float cutoff; // 0.0 - 1.0 (freq de corte normalizada)
    float resonance; // 0.0 - 1.0
    float vcfDecay; // ms, 20 - 2000
    float vcfSustain; // 0.0 - 1.0
    float vcaDecay; // ms
    float accentAmt; // 0.0 - 1.0
    float slideTime; // ms, 0 - 500
    float volume; // 0.0 - 1.0
    float lfoRate; // Hz, 0.01 - 20
    float lfoDepth; // 0.0 - 1.0
    WaveShape wave;
};

```

```

class Synth {
public:
    SynthParams params;
    void init();
    void noteOn(uint8_t note, bool accent, bool slide);
    void noteOff(uint8_t note);
    void process(int16_t* buffer, int numSamples);
private:
    // Wavetables
    float sawTable[WAVETABLE_SIZE];
    float sqrTable[WAVETABLE_SIZE];
    float triTable[WAVETABLE_SIZE];
    // Oscillator state
    float phase;
    float phaseInc;
    float currentFreq;
    float targetFreq;
    float slideRate;
    // Sub-oscillator
    float subPhase;
    // Filter state (Huovilainen ladder)
    float stage[4], delay[4];
    float cutoffSmooth;
    // Envelopes
    float vcfEnv, vcaEnv;
    float vcfEnvDecay, vcaEnvDecay;
    float accentEnv;
    // LFO
    float lfoPhase;
    // Utility
    void buildWavetables();
    float noteToFreq(uint8_t note);
    float processFilter(float input);
};

```

5.4 synth.cpp — Motor de síntesis

```

#include "synth.h"
#include <string.h>
#include <math.h>

void Synth::init() {
    memset(stage, 0, sizeof(stage));
    memset(delay, 0, sizeof(delay));
    phase = subPhase = lfoPhase = 0.0f;
    vcfEnv = vcaEnv = accentEnv = 0.0f;
    currentFreq = targetFreq = 440.0f;
    cutoffSmooth = 0.3f;
    // Defaults sonido acid clásico
    params.cutoff = 0.25f;
    params.resonance = 0.75f;
    params.vcfDecay = 200.0f;
    params.vcfSustain = 0.05f;
    params.vcaDecay = 500.0f;
    params.accentAmt = 0.8f;
    params.slideTime = 60.0f;
    params.volume = 0.7f;
    params.lfoRate = 0.5f;
    params.lfoDepth = 0.0f;
    params.wave = WAVE_SAW;
    buildWavetables();
}

```

```

void Synth::buildWavetables() {
    for (int i = 0; i < WAVETABLE_SIZE; i++) {
        float t = (float)i / WAVETABLE_SIZE;
        // Sawtooth: suma de armónicos (band-limited, 8 parciales)
        sawTable[i] = 0;
        for (int h = 1; h <= 8; h++)
            sawTable[i] += sinf(TWO_PI * h * t) / h;
        sawTable[i] *= 0.4f; // normalizar
        // Square: solo armónicos impares
        sqrTable[i] = 0;
        for (int h = 1; h <= 8; h += 2)
            sqrTable[i] += sinf(TWO_PI * h * t) / h;
        sqrTable[i] *= 0.5f;
        // Triangle
        triTable[i] = t < 0.5f ? 4.0f*t - 1.0f : 3.0f - 4.0f*t;
    }
}

float Synth::noteToFreq(uint8_t note) {
    return 440.0f * powf(2.0f, (note - 69) / 12.0f);
}

void Synth::noteOn(uint8_t note, bool accent, bool slide) {
    targetFreq = noteToFreq(note);
    if (!slide) currentFreq = targetFreq;
    slideRate = (params.slideTime < 1.0f) ? 1.0f :
        expf(-1.0f / (params.slideTime * 0.001f * SAMPLE_RATE));
    // Trigger envelopes
    vcfEnv = 1.0f;
    vcaEnv = 1.0f;
    accentEnv = accent ? 1.0f : 0.0f;
    vcfEnvDecay = expf(-1.0f / (params.vcfDecay * 0.001f * SAMPLE_RATE));
    vcaEnvDecay = expf(-1.0f / (params.vcaDecay * 0.001f * SAMPLE_RATE));
}

void Synth::noteOff(uint8_t note) {
    vcaEnvDecay = expf(-1.0f / (50.0f * 0.001f * SAMPLE_RATE)); // quick release
}

float Synth::processFilter(float input) {
    // Huovilainen Moog Ladder Filter
    float fc = cutoffSmooth * 0.5f;
    float f = fc * 1.16f;
    float k = params.resonance * 3.98f;
    float p = f * (1.8f - 0.8f * f);
    float q = k * (1.0f + 0.5f * p * (1.0f - p));
    input -= q * stage[3];
    // Tanh saturation (aproximación rápida)
    input = input / (1.0f + fabsf(input));
    stage[0] = input * p + delay[0] * (1.0f - p);
    stage[1] = stage[0]*p + delay[1]*(1.0f - p);
    stage[2] = stage[1]*p + delay[2]*(1.0f - p);
    stage[3] = stage[2]*p + delay[3]*(1.0f - p);
    delay[0] = input; delay[1] = stage[0];
    delay[2] = stage[1]; delay[3] = stage[2];
    return stage[3];
}

void Synth::process(int16_t* buffer, int numSamples) {
    float invSR = 1.0f / SAMPLE_RATE;
    for (int i = 0; i < numSamples; i++) {
        // — Portamento —
        currentFreq = targetFreq + (currentFreq - targetFreq) * slideRate;
        phaseInc = currentFreq * WAVETABLE_SIZE * invSR;
    }
}

```

```

// — Oscilador principal —
int idx = (int)phase & (WAVETABLE_SIZE - 1);
float osc = (params.wave == WAVE_SAW) ? sawTable[idx] :
            (params.wave == WAVE_SQUARE) ? sqrTable[idx] : triTable[idx];
phase += phaseInc;
if (phase >= WAVETABLE_SIZE) phase -= WAVETABLE_SIZE;
// — Sub-oscilador (octava abajo, square) —
int subIdx = (int)subPhase & (WAVETABLE_SIZE - 1);
float sub = sqrTable[subIdx] * 0.3f;
subPhase += phaseInc * 0.5f;
if (subPhase >= WAVETABLE_SIZE) subPhase -= WAVETABLE_SIZE;
// — LFO —
float lfo = sinf(lfoPhase) * params.lfoDepth;
lfoPhase += TWO_PI * params.lfoRate * invSR;
if (lfoPhase > TWO_PI) lfoPhase -= TWO_PI;
// — Cutoff dinámico —
float envCutoff = params.cutoff + vcfEnv * 0.6f + accentEnv * params.accentAmt *
0.4f + lfo * 0.2f;
envCutoff = fmaxf(0.02f, fminf(0.98f, envCutoff));
// Smooth cutoff (evitar zipper noise)
cutoffSmooth += (envCutoff - cutoffSmooth) * 0.01f;
// — Filtro —
float filtered = processFilter(osc + sub);
// — VCA —
float amp = vcaEnv * params.volume * (1.0f + accentEnv * 0.5f);
float out = filtered * amp;
// — Actualizar envelopes —
vcfEnv = params.vcfSustain + (vcfEnv - params.vcfSustain) * vcfEnvDecay;
vcaEnv *= vcaEnvDecay;
accentEnv *= 0.9995f;
// — Clamp y convertir a 16-bit —
out = fmaxf(-1.0f, fminf(1.0f, out));
int16_t sample = (int16_t)(out * 32767.0f);
buffer[i*2] = sample; // Left
buffer[i*2+1] = sample; // Right
}
}

```

5.5 sequencer.h / sequencer.cpp

```

// sequencer.h
#pragma once
#include "synth.h"

#define NUM_STEPS 16

struct Step {
    uint8_t note; // 0-127 MIDI note
    bool active; // si suena
    bool accent; // accent activado
    bool slide; // slide al siguiente
    uint8_t octave; // 0=bajo, 1=medio, 2=alto
};

class Sequencer {
public:
    Step steps[NUM_STEPS];
    bool playing;
    int bpm;
    int currentStep;
    Synth* synth;
    void init(Synth* s);
    void update();

```

```

void play();
void stop();
void setStep(int i, uint8_t note, bool active, bool accent, bool slide);
void loadAcidPattern(); // patrón demo acid
private:
    unsigned long lastStepTime;
    unsigned long stepInterval; // microsegundos
};

// sequencer.cpp
#include "sequencer.h"

void Sequencer::init(Synth* s) {
    synth = s; playing = false; currentStep = 0; bpm = 130;
    stepInterval = 60000000UL / (bpm * 4); // corcheas a 4/4
    lastStepTime = 0;
    loadAcidPattern();
}

void Sequencer::loadAcidPattern() {
    // Patrón acid clásico en C (nota 36 = C2)
    uint8_t notes[] = {36,36,48,36, 39,36,43,36, 36,48,36,43, 41,36,43,48};
    bool acc[] = {1, 0, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1};
    bool sld[] = {0, 1, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0};
    for (int i = 0; i < 16; i++)
        setStep(i, notes[i], true, acc[i], sld[i]);
}

void Sequencer::setStep(int i, uint8_t note, bool active, bool accent, bool slide) {
    steps[i].note = note; steps[i].active = active;
    steps[i].accent = accent; steps[i].slide = slide;
}

void Sequencer::update() {
    if (!playing) return;
    unsigned long now = micros();
    if (now - lastStepTime >= stepInterval) {
        lastStepTime = now;
        Step& s = steps[currentStep];
        if (s.active) {
            bool nextSlide = steps[(currentStep + 1) % NUM_STEPS].slide;
            synth->noteOn(s.note, s.accent, s.slide);
        }
        currentStep = (currentStep + 1) % NUM_STEPS;
    }
}

void Sequencer::play() { playing = true; currentStep = 0; }
void Sequencer::stop() { playing = false; synth->noteOff(0); }

```

6. Interfaz de Usuario (UI)

6.1 ui.h / ui.cpp — OLED + Potenciómetros

```
// ui.h
#pragma once
#include "synth.h"
#include "sequencer.h"
#include <Adafruit_SSD1306.h>

#define OLED_ADDR 0x3C
#define OLED_WIDTH 128
#define OLED_HEIGHT 64

// Pines ADC de potenciómetros
#define POT_CUTOFF 34
#define POT_RESONANCE 35
#define POT_DECAY 32
#define POT_ACCENT 33
#define POT_VOLUME 36
#define POT_WAVE 39
#define POT_LFO_RATE 4
#define POT_SLIDE 2

// Pines botones
#define BTN_PLAY 5
#define ENC_CLK 18
#define ENC_DT 17
#define ENC_SW 16

class UI {
public:
    void init(Synth* s, Sequencer* seq);
    void update();
private:
    Synth* synth;
    Sequencer* seq;
    Adafruit_SSD1306 display;
    void readPots();
    void readButtons();
    void updateDisplay();
    int smoothADC(int pin, int& last);
    int lastPotValues[8];
};

// ui.cpp
#include "ui.h"

void UI::init(Synth* s, Sequencer* sq) {
    synth = s; seq = sq;
    display = Adafruit_SSD1306(OLED_WIDTH, OLED_HEIGHT, &Wire, -1);
    Wire.begin(21, 19); // SDA=21, SCL=19
    display.begin(SSD1306_SWITCHCAPVCC, OLED_ADDR);
    display.clearDisplay();
    display.setTextColor(SSD1306_WHITE);
    pinMode(BTN_PLAY, INPUT_PULLUP);
    memset(lastPotValues, -1, sizeof(lastPotValues));
}

int UI::smoothADC(int pin, int& last) {
    int raw = analogRead(pin);
    // Media exponencial + histeresis
    int smooth = last < 0 ? raw : (last * 7 + raw) / 8;
```

```

    last = smooth;
    return smooth;
}

void UI::readPots() {
    synth->params.cutoff    = smoothADC(POT_CUTOFF, lastPotValues[0]) / 4095.0f;
    synth->params.resonance = smoothADC(POT_RESONANCE, lastPotValues[1]) / 4095.0f;
    float d = smoothADC(POT_DECAY, lastPotValues[2]) / 4095.0f;
    synth->params.vcfDecay   = 20.0f + d * 1980.0f; // 20-2000ms
    synth->params.accentAmt = smoothADC(POT_ACCENT, lastPotValues[3]) / 4095.0f;
    synth->params.volume    = smoothADC(POT_VOLUME, lastPotValues[4]) / 4095.0f;
    int w = smoothADC(POT_WAVE, lastPotValues[5]) / 4095.0f * 3;
    synth->params.wave      = (WaveShape)constrain(w, 0, 2);
    float lr = smoothADC(POT_LFO_RATE, lastPotValues[6]) / 4095.0f;
    synth->params.lfoRate   = 0.1f + lr * 19.9f; // 0.1-20 Hz
    float sl = smoothADC(POT_SLIDE, lastPotValues[7]) / 4095.0f;
    synth->params.slideTime = sl * 300.0f; // 0-300ms
}

void UI::readButtons() {
    static bool lastPlay = HIGH;
    bool play = digitalRead(BTN_PLAY);
    if (play == LOW && lastPlay == HIGH) {
        if (seq->playing) seq->stop(); else seq->play();
        delay(50); // debounce
    }
    lastPlay = play;
}

void UI::updateDisplay() {
    static unsigned long lastRefresh = 0;
    if (millis() - lastRefresh < 100) return; // 10fps
    lastRefresh = millis();
    display.clearDisplay();
    // Header
    display.setTextSize(1);
    display.setCursor(0,0);
    display.print(seq->playing ? '> PLAY' : ' STOP');
    display.setCursor(70,0);
    display.print('BPM:'); display.print(seq->bpm);
    // Parámetros sintetizador
    display.setCursor(0,16);
    display.print('CUT:'); display.print((int) (synth->params.cutoff*100));
    display.print(' RES:'); display.print((int) (synth->params.resonance*100));
    display.setCursor(0,26);
    display.print('DEC:'); display.print((int) synth->params.vcfDecay);
    display.print('ms ACC:'); display.print((int) (synth->params.accentAmt*100));
    // Steps display
    for (int i = 0; i < 16; i++) {
        int x = (i % 8) * 15 + 2;
        int y = (i < 8) ? 42 : 54;
        bool isActive = seq->steps[i].active;
        bool isCurrent = (i == seq->currentStep) && seq->playing;
        if (isCurrent) display.fillRect(x, y, 12, 8, SSD1306_WHITE);
        else if (isActive) display.drawRect(x, y, 12, 8, SSD1306_WHITE);
        else display.drawRect(x, y, 12, 8, SSD1306_WHITE);
        if (seq->steps[i].accent && !isCurrent)
            display.fillRect(x+4, y+2, 4, 4, SSD1306_WHITE);
    }
    display.display();
}

void UI::update() {
    readPots();
    readButtons();
    updateDisplay();
}

```


}

7. Guía de Construcción Paso a Paso

7.1 Configurar el entorno de desarrollo

- Instala Arduino IDE 2.x desde arduino.cc
- Agrega ESP32 al Boards Manager: File → Preferences → Additional Boards Manager URLs
- URL: https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json
- Tools → Board → Boards Manager → buscar 'esp32' → instalar 'esp32 by Espressif Systems'
- Instala librerías: Sketch → Include Library → Manage Libraries → buscar e instalar: 'Adafruit SSD1306' y 'Adafruit GFX Library'
- Selecciona placa: Tools → Board → ESP32 Arduino → 'ESP32 Dev Module'
- CPU Frequency: 240MHz | Flash Size: 4MB | Partition Scheme: Default

7.2 Construcción del hardware

Fase 1: Preparación

- Planifica el layout en la PCB perforada antes de soldar
- Ubica el ESP32 en el centro, el DAC PCM5102A a la derecha, OLED arriba
- Los potenciómetros van en el panel frontal (placa separada o misma PCB)
- Usa diferentes colores de cable: rojo=VCC, negro=GND, amarillo=señal, azul=I2C, verde=I2S

Fase 2: Alimentación

- Conecta primero toda la red de alimentación y GND
- Verifica con multímetro que 3.3V y GND no tienen corto
- Agrega condensadores de bypass (100nF) cerca de cada IC

Fase 3: I2S y DAC

- Suelta el módulo PCM5102A
- Conecta LCK→GPIO25, BCK→GPIO26, DIN→GPIO22
- Conecta SCK del PCM5102A a GND
- Conecta XSMT a 3.3V (deshabilitar mute)
- Instala resistencias de 33Ω en LOUT y ROUT antes del jack de audio

✓ Prueba el DAC primero con un sketch simple de I2S que genere un tono sinusoidal antes de integrar el código completo

Fase 4: Display OLED

- Conecta SDA→GPIO21, SCL→GPIO19
- VCC→3.3V, GND→GND
- Agrega resistencias pull-up de 4.7kΩ en SDA y SCL si el módulo no las incluye

Fase 5: Potenciómetros

- Conecta el pin central (wiper) de cada pot al GPIO ADC correspondiente
- Pin 1 → 3.3V, Pin 3 → GND (o viceversa para invertir dirección)
- Agrega condensadores de 100nF entre wiper y GND para filtrar ruido

Fase 6: Botones y LEDs

- Botones: un terminal a GPIO, otro a GND. El ESP32 usa INPUT_PULLUP interno
- LEDs: Ánodo → GPIO via resistencia 220Ω → Cátodo → GND

7.3 Primera prueba y calibración

```
// Test 1: Verificar I2S - sube este sketch primero
#include "driver/i2s.h"

void setup() {
    i2s_config_t cfg = {
        .mode=(i2s_mode_t)(I2S_MODE_MASTER|I2S_MODE_TX),
        .sample_rate=44100, .bits_per_sample=I2S_BITS_PER_SAMPLE_16BIT,
        .channel_format=I2S_CHANNEL_FMT_RIGHT_LEFT,
        .communication_format=I2S_COMM_FORMAT_STAND_I2S,
        .dma_buf_count=4, .dma_buf_len=256, .use_apll=true
    };
    i2s_pin_config_t pins = {.bck_io_num=26, .ws_io_num=25, .data_out_num=22};
    i2s_driver_install(I2S_NUM_0, &cfg, 0, NULL);
    i2s_set_pin(I2S_NUM_0, &pins);
}

void loop() {
    static float phase=0;
    int16_t buf[512];
    for(int i=0; i<256; i++){
        int16_t s=(int16_t)(sinf(phase)*20000);
        buf[i*2]=buf[i*2+1]=s;
        phase+=2*3.14159f*440/44100;
    }
    size_t bw; i2s_write(I2S_NUM_0, buf, sizeof(buf), &bw, portMAX_DELAY);
}
```

8. Guía de Sonido ACID — Parámetros y Técnicas

8.1 Configuración base del sonido 303 clásico

Parámetro	Valor típico 303	Descripción
Waveform	Sawtooth	El 303 usa saw para el sonido más rico
Cutoff	25–40%	Comienza cerrado, la envolvente lo abre
Resonance	65–85%	El pico de resonancia es la firma del acid
VCF Decay	100–250 ms	Corto = más percusivo/acid, largo = suave
VCF Sustain	0–10%	Casi siempre bajo para el efecto de cierre
Accent Level	70–90%	Accent es lo que hace el 'squeak'
Slide Time	40–80 ms	El glide suave entre notas deslizadas
Tempo	130–145 BPM	Acid techno clásico

8.2 Progresión para lograr el sonido acid paso a paso

Sigue este orden para encontrar el sonido acid en tu sintetizador:

- ⚡ 1. PON: Cutoff al 20%, Resonance al 75%, VCF Decay al 150ms, Accent al 85%
2. Carga el patrón demo (loadAcidPattern()) o programa un patrón simple
3. Dale PLAY a 130 BPM — deberías escuchar el squelch ácido básico
4. Sube lentamente el Cutoff a ~35% — el filtro empieza a 'cantar'
5. Sube Resonance cerca del 85–90% — aumenta el pico del filtro
6. Ajusta VCF Decay: más corto (80ms) = más 'clank', más largo (400ms) = más 'wob'
7. Activa Accent en los pasos marcados — deberían sonar más 'abiertos' y fuertes
8. Activa Slide en algunos steps — el portamento crea el efecto glide característico

8.3 Patrones rítmicos acid clásicos

```
// Patrón 1: Acid House básico (nota C, C#, F, G)
// Steps: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
16
// Nota: C2 C2 C#2 C2 F2 C2 G2 C2 C2 C#2 C2 G2 F2 C2 G2
C#2
// Accent: X . . . X . X . . X . . X . .
X
// Slide: . X . . . X . . X . X . . . X
.

// Patrón 2: Acid Techno agresivo
// Nota: C2 C2 . C2 C#2 . C2 G1 C2 C2 . C#2 F2 . G1
C2
// Accent: X . . . X . . X . . . X . . X
X
```

```
// Slide: . X . . . . X . . X . . X . .  
.
```

8.4 Técnicas avanzadas

Filter Sweep en tiempo real

Durante el performance, gira el potenciómetro de Cutoff lentamente de 0 a 100% y de vuelta mientras el secuenciador corre. Este es el efecto más icónico del acid: el filtro abriendo y cerrando.

Resonance Push

Sube la resonancia a 90–95% — el filtro comenzará a auto-oscilar, generando una frecuencia de tono puro. Esto añade una nota de 'silbido' sobre la música. El 303 original era famoso por esto.

Tempo modulation

Cambia el BPM con el encoder durante el playback. Pasar de 130 a 145 BPM en el drop da energía extra al patrón.

Accent density

Más accent = sonido más abierto y 'screaming'. Para acid suave, accent en 1 de cada 4 steps. Para acid agresivo, accent en la mitad de los steps.

9. Implementación MIDI

9.1 midi.h / midi.cpp

```
// midi.h
#pragma once
#include "synth.h"
#include "sequencer.h"

class MidiHandler {
public:
    void init(Synth* s, Sequencer* sq);
    void update();
private:
    Synth* synth; Sequencer* seq;
    uint8_t midiBuffer[3];
    int bufIdx;
    void handleMessage(uint8_t status, uint8_t d1, uint8_t d2);
};

// midi.cpp
#include "midi.h"

void MidiHandler::init(Synth* s, Sequencer* sq) {
    synth = s; seq = sq; bufIdx = 0;
    Serial.begin(31250); // MIDI baud rate
}

void MidiHandler::update() {
    while (Serial.available()) {
        uint8_t byte = Serial.read();
        if (byte & 0x80) { bufIdx = 0; midiBuffer[bufIdx++] = byte; }
        else if (bufIdx > 0 && bufIdx < 3) { midiBuffer[bufIdx++] = byte; }
        if (bufIdx == 3) {
            handleMessage(midiBuffer[0], midiBuffer[1], midiBuffer[2]);
            bufIdx = 0;
        }
    }
}

void MidiHandler::handleMessage(uint8_t status, uint8_t d1, uint8_t d2) {
    uint8_t cmd = status & 0xF0;
    if (cmd == 0x90 && d2 > 0) // Note On
        synth->noteOn(d1, d2 > 100, false); // velocity > 100 = accent
    else if (cmd == 0x80 || (cmd == 0x90 && d2 == 0)) // Note Off
        synth->noteOff(d1);
    else if (cmd == 0xB0) { // Control Change
        switch(d1) {
            case 74: synth->params.cutoff = d2/127.0f; break; // CC74 = Cutoff
            case 71: synth->params.resonance = d2/127.0f; break; // CC71 = Resonance
            case 75: synth->params.vcfDecay = 20+d2*15.6f; break;
            case 7: synth->params.volume = d2/127.0f; break; // CC7 = Volume
        }
    }
    else if (cmd == 0xF8) { // MIDI Clock
        // Implementar sync MIDI externo aquí
    }
}
```

⚡ MIDI CC74 (Cutoff), CC71 (Resonance) y CC7 (Volume) son los CCs estándar para estos parámetros. La mayoría de DAWs y controladores los usan por defecto.

9.2 Extensiones opcionales

- MIDI Clock Sync: sincronizar el secuenciador al MIDI clock del DAW (mensaje 0xF8 cada 24 pulsos/beat)
- USB MIDI: con la librería 'USB-MIDI' el ESP32-S2/S3 puede aparecer como dispositivo MIDI USB nativo
- MIDI Over BLE: con ESP32 y la librería 'BLE-MIDI' para control inalámbrico desde iPad/iPhone

10. Troubleshooting y Problemas Comunes

Síntoma	Causa probable	Solución
No hay audio	I2S mal configurado o DAC sin alimentación	Verifica XSMT=3.3V en PCM5102A; testa con el sketch de prueba I2S
Audio con clicks/glitches	Buffer underrun en el DMA	Aumenta dma_buf_count a 8-16; verifica que audioTask está en Core 0
Sonido muy distorsionado	Clipping en el filtro con resonancia alta	Baja el volumen de entrada al filtro; ajusta el gain de la tanh()
Potenciómetros ruidosos	ADC del ESP32 sensible al ruido	Agrega 100nF cap en cada wiper; usa smoothADC() con más promediado
No se ve el OLED	Dirección I2C incorrecta	Corre un I2C Scanner sketch para encontrar la dirección (0x3C o 0x3D)
Secuenciador fuera de tiempo	micros() overflow o delay en loop	Usa tareas FreeRTOS separadas; nunca uses delay() en la tarea del seq.
MIDI no responde	Baud rate incorrecto o circuito optoacoplador	MIDI usa 31250 baud. Verifica el optoacoplador con osciloscopio
Filter no hace 'squeak'	Resonance muy baja o cutoff muy alto	Cutoff < 30%, Resonance > 70%, VCF Decay < 200ms

10.1 Notas sobre el ADC del ESP32

El ADC del ESP32 tiene problemas conocidos de no-linealidad, especialmente en los extremos (0-100 mV y 3100-3300 mV). Para mejores resultados:

- Usa atenuación ADC_ATTEN_DB_11 para rango completo de 0-3.3V
- Agrega condensadores de 100nF en las entradas ADC
- Implementa una tabla de corrección (espressif provee una en esp_adc_cal)
- Filtra con media móvil exponencial (EMA) para suavizar el ruido

```
#include <esp_adc_cal.h>
esp_adc_cal_characteristics_t adc_chars;
// En setup():
esp_adc_cal_characterize(ADC_UNIT_1, ADC_ATTEN_DB_11, ADC_WIDTH_BIT_12,
                        1100, &adc_chars);
// Para leer con corrección:
uint32_t voltage = esp_adc_cal_raw_to_voltage(analogRead(34), &adc_chars);
// voltage está en mV (0-3300)
```


11. Diseño de Enclosure (Carcasa)

11.1 Dimensiones recomendadas

Dimensión	Medida	Notas
Largo	220 mm	Suficiente para 16 botones de step en fila
Ancho	130 mm	2 filas de pots + espacio para display
Alto	50 mm	Suficiente para componentes THT
Inclinación frontal	10-15°	Mejora ergonomía en mesa (opcional)

11.2 Layout del panel frontal

[OLED 128x64]ACID SYNTH ESP32[PLAY]

[CUT] [RES] [DEC] [ACC] [VOL] [WAVE] [LFO] [SLD]

[1] [2] [3] [4] [5] [6] [7] [8]

[9] [10] [11] [12] [13] [14] [15] [16]

o AUDIO o MIDI

Leyenda:

[CUT] = Potenciómetro Cutoff

[RES] = Resonance

[DEC] = VCF Decay

[1-16] = Botones de step + LED

[ENCDR] = Encoder BPM

o AUDIO = Jack 3.5mm estéreo

o MIDI = TRS MIDI o DIN5

11.3 Impresión 3D

El diseño de carcasa puede imprimirse en FDM con PLA o PETG. Parámetros sugeridos:

- Material: PLA negro o PETG (más resistente al calor)
- Capa: 0.2mm, Relleno: 20% gyroid
- Perforaciones para potenciómetros: 7mm diámetro
- Perforaciones para botones 12mm: 12mm diámetro
- Ranura OLED: 26mm × 14mm
- Ventilación inferior: rejilla de ranuras 1mm, si hay generación de calor

✓ Los archivos STL de diseño de carcasa para este proyecto pueden crearse con OpenSCAD. Los parámetros del diseño están en el archivo enclosure_params.scad.

12. Mejoras y Próximos Pasos

12.1 Mejoras de hardware

- PCB diseñada a medida (KiCad): elimina el cableado manual y mejora la confiabilidad
- ESP32-S3 (en lugar de ESP32 original): más RAM, mejor ADC, soporte USB MIDI nativo
- Pantalla TFT color 240×320: para visualizar la forma de onda en tiempo real
- Filtro analógico externo (CEM3320 o IR3109): para mayor autenticidad
- CVout: convertidor I2C-DAC (MCP4725) para salida de voltaje de control a sintetizadores analógicos

12.2 Mejoras de firmware

- Overdrive/Distortion DSP: añadir saturación waveshaper después del filtro
- Reverb/Delay: implementar delay con buffer en PSRAM del ESP32
- Guardado de patrones en SPIFFS: guardar hasta 16 patrones en memoria flash
- Arpeggiator: modo automático que genera patrones desde acordes MIDI
- Polysynth: usar FreeRTOS para 4 voces simultáneas (polifonía)

12.3 Recursos adicionales

- musicdsp.org — catálogo de algoritmos DSP para audio, incluyendo filtros y efectos
- Repositorio de referencia: github.com/perkowitz/acid-step-sequencer
- 'The Art of VA Filter Design' por Vadim Zavalishin (PDF gratuito) — teoría profunda de filtros VA
- Will Pirkle 'Designing Software Synthesizer Plug-Ins in C++' — libro referencia de síntesis
- Foro esp32.com — soporte de la comunidad ESP32

□ **¡Buena suerte con tu build! El sonido acid te espera.**

Cualquier duda, el 303 original costaba \$395 en 1981. El tuyo costará \$35 y lo hiciste tú.